

const, static & constexpr

Junior / Mid C++ հարցազրույցի խորացված ուղեցույց

Այս մասը ընդգրկում է `const` (variable, parameter, member function, pointer-ի բոլոր տարբերակները), `static` (local, class member, member function), `constexpr`, lifetime-ի տարբերությունները և lvalue/rvalue overloading-ի «թակարդ» հարցերը՝ պատրաստի interview պատասխաններով:

1. const-ի հիմնական իմաստը

```
const int x = 10;
```

Նշանակում է՝ **x-ը կարգավ կարելի է, փոխել՝ ոչ:**

```
cout << x;    // OK
x = 20;       // ERROR
```

const function parameter

```
void print(const string& s) { }
```

Interview-ներում շատ են տալիս՝ ինչո՞ւ `const string&`:

1. Copy չի լինում (reference է)
2. Function-ը չի կարող փոխել object-ը (const է)

```
void print(string& s)      { s = "abc"; }    // կարող է փոխել
void print(const string& s) { s = "abc"; }    // ERROR
```

const member function

```
class Person {
public:
    int age;

    int getAge() const {    // const -> չի փոխի object-ը
        return age;        // OK
        // age = 30;       // ERROR
    }
};
```

Վերջի `const`-ը վերաբերում է object-ին և ասում է՝ «այս function-ը չի փոխի object-ը»:

2. const և Pointer-ներ

Կարդա աջից ձախ, և հասկանալը հեշտ է դառնում:

const int* ptr — տվյալն է const

```
int x = 10;
const int* ptr = &x;    // ptr -> const int

ptr = &y;    // OK (pointer-ը կարող է փոխվել)
*ptr = 20;  // ERROR (տվյալը չի կարող փոխվել)
```

int* const ptr — pointer-ն է const

```
int x = 10;
int* const ptr = &x;    // const pointer to int

*ptr = 20;    // OK (տվյալը կարող է փոխվել)
ptr = &y;    // ERROR (pointer-ը չի կարող փոխվել)
```

const int* const ptr — երկուսն էլ const

```
const int* const ptr = &x;

*ptr = 20;    // ERROR
ptr = &y;    // ERROR
```

Interview պատասխան

```
const int* p    -> տվյալը const է
int* const p    -> pointer-ն է const
```

const prevents modification of an object after initialization and helps write safer code.

3. static

static local variable

Սովորական local variable-ը ամեն կանչում նորից է ստեղծվում.

```
void foo() {  
    int x = 0;  
    x++;  
    cout << x << endl;  
}  
  
// foo(); foo(); foo(); -> 1 1 1
```

static local variable-ը ստեղծվում է միայն մեկ անգամ և պահպանում է արժեքը.

```
void foo() {  
    static int x = 0;  
    x++;  
    cout << x << endl;  
}  
  
// foo(); foo(); foo(); -> 1 2 3
```

Lifetime-ը՝ ստեղծվում է առաջին կանչի ժամանակ, մահանում է program end-ին:

static class member

```
class Person {  
public:  
    static int count;  
  
    Person() { count++; }  
};  
  
int Person::count = 0; // definition (class-ից դուրս)  
  
// Person p1, p2, p3;  
// cout << Person::count; -> 3
```

Բոլոր object-ների համար կա միայն մեկ `count`՝ ոչ թե `p1.count/p2.count/p3.count`, այլ `Person::count`: Այն պատկանում է class-ին, ոչ թե object-ին:

static member function

```
class Person {  
public:  
    static void hello() { cout << "Hello"; }  
};
```

```
Person::hello();    // object պետք չէ
```

Ինչու static function-ը չի տեսնում non-static member-ը

```
class Person {  
public:  
    int age;  
    static void print() {  
        cout << age;    // ERROR  
    }  
};
```

`age` -ը կապված է կոնկրետ object-ի հետ, իսկ static function-ը object չունի, չկա `this` pointer: Դրա համար static function-ը կարող է աշխատել միայն static member-ների հետ:

4. constexpr

`constexpr` -ը հայտնվել է C++11-ում և ասում է compiler-ին՝ եթե հնարավոր է, հաշվիր այս արժեքը **compile time**-ում:

const vs constexpr

```
const int x = 10;    // immutable, բայց պարտադիր չէ compile-time հայտնի լինի
```

```
int foo() { return 10; }
```

```
const int x = foo();    // OK (runtime)
```

```
constexpr int y = foo(); // ERROR (foo()-ը compile-time չի հաշվարկվում)
```

constexpr function

```
constexpr int square(int x) {  
    return x * x;  
}
```

```
constexpr int a = square(5);    // compiler-ը հաշվում է 25 մինչև run
```

Interview պատասխան

```
const    -> immutable object  
constexpr -> compile-time constant
```

5. const-ի optimization-ները

Optimization #1 — constant folding.

```
const int x = 10;
int y = x + 5;      // compiler-ը կարող է դարձնել int y = 15;
```

Optimization #2 — `constexpr int x = 10;` գրեթե երաշխավորված compile-time evaluation է:

Optimization #3 — const reference-ը խնայում է copy-ն.

```
void print(const string& s) { }    // copy չկա
void print(string s)              { } // string copy: allocation + copy ctor + extra CPU
```

6. lvalue / rvalue Overloading թեստեր

```
void f(int&) { cout << "lvalue\n"; }
void f(int&&) { cout << "rvalue\n"; }
```

Կանչ	Output	Ինչու
<code>int x=5; f(x);</code>	lvalue	<code>x</code> -ը lvalue է
<code>f(5);</code>	rvalue	<code>5</code> -ը rvalue է
<code>f(x + 1);</code>	rvalue	<code>x+1</code> -ը temporary է
<code>int&& r=5; f(r);</code>	lvalue	<code>r</code> -ը անուն ունի -> lvalue
<code>f(std::move(r));</code>	rvalue	cast to rvalue

Ամենակարևոր կանոնը՝ **Any named variable is an lvalue**: `int&& r = 5;` անելուց հետո `r` -ն ինքը lvalue է, թեև rvalue reference type ունի:

const-ի հետ

```
const int x = 5;
f(x); // ERROR, եթե միայն f(int&) և f(int&&) կան
```

`const int` -ը չի bind լինում `int&` -ին: Պետք է ավելացնել.

```
void f(const int&) { cout << "const lvalue"; }
// հիմա const int x=5; f(x); -> const lvalue
```

Մեծ թեստ

```
void f(int&)          { cout << "A\n"; }
void f(const int&)    { cout << "B\n"; }
void f(int&&)         { cout << "C\n"; }

int main() {
    int x = 10;
    const int y = 20;

    f(x);             // A
    f(y);             // B
    f(30);            // C
    f(x + y);         // C
    f(std::move(x));  // C
}
```

Output' A B C C C :

7. Lifetime — const vs static

Normal / const local variable

```
void foo() {  
    int x = 5;           // ստեղծ.՝ function enter, մահ.՝ function exit  
    const int y = 5;     // ՆՈՒՅՆ lifetime-ը  
}
```

Շատ կարևոր՝ **const-ը lifetime-ը չի փոխում**: `const != static`:

static local variable

```
void foo() {  
    static int x = 5;    // ստեղծ.՝ առաջին կանչ, մահ.՝ program end  
}
```

static class member

```
class Test {  
public:  
    static int count;  
};  
  
int Test::count = 0;    // Program Start -> Program End, մեկ instance
```

Խաբուսիկ հարց — static object-ի destructor

```
class Test {
public:
    Test() { cout << "ctor\n"; }
    ~Test() { cout << "dtor\n"; }
};

void foo() { static Test t; }

foo();
cout << "middle\n";
// Output:
// ctor
// middle
// dtor      <- destructor-ը կանչվում է program exit-ի ժամանակ
```

Ամփոփ Cheat Sheet

const — չի փոխում lifetime-ը, պաշտպանում է object-ը, կարող է օգնել optimization-ին (constant folding, no-copy references):

constexpr — compile-time evaluation. `const` = immutable object, `constexpr` = compile-time constant.

const int* vs **int* const** — առաջինում տվյալն է const, երկրորդում՝ pointer-ը (կարդա աջից ձախ):

const member function — `T f() const;` երաշխավորում է, որ object-ը չի փոխվի; չունի իրավունք գրելու non-mutable member-ներ:

static local — ստեղծվում է մեկ անգամ, ապրում մինչև program end.

static member — պատկանում է class-ին, ոչ object-ին; մեկ instance.

static function — չունի `this`, աշխատում է միայն static member-ների հետ.

Named rvalue reference — `int&& r = 5;` -> `r` -ը օգտագործելիս lvalue է (famous trap).

std::move — չի move անում, ուղղակի cast է `T&&`.

OneMarketData takeaways

<code>const</code>	-> չի փոխում lifetime, պաշտպանում է object-ը, օգնում optimization-ին
<code>constexpr</code>	-> compile-time evaluation
<code>static</code>	-> lifetime extends to program termination
<code>std::move</code>	-> չի move անում, cast է T&&
<code>int&& r=5</code>	-> r-ը օգտագործելիս lvalue է

Սրանից հետո սովորաբար անցնում են `Rule of Three`, `Rule of Five`, `unique_ptr`, `shared_ptr`, `virtual functions` և `vtable` թեմաներին: